Temas que se evalúan en el Segundo parcial

Unidad 2: Gestión Lean Agile de Productos de Software

- Framework para escalar SCRUM / Nexus
- Filosofía Lean - Método Kanban en el contexto del desarrollo de software
- Métricas en los 3 enfoques (Tradicional / Lean / Agile)

Unidad 3: Gestión de software como producto

- Prácticas continuas

Unidad 4: Aseguramiento de Calidad de Proceso y de Producto

Contenidos:

- Conceptos generales sobre calidad.
- Importancia de trabajar para y con Calidad. Ventajas y Desventajas.
- Actividades relacionadas con el Aseguramiento de la Calidad del Software.
- Principales Modelos de Calidad existentes (CMMI – SPICE – ISO).
- Auditorías al Software: Auditorías de Proyecto y Auditorías al Grupo de Calidad.
- Proceso de Auditorías
- Calidad de Producto: Planificación de pruebas para el software- Niveles y tipos de pruebas para el software. Técnicas y herramientas para probar software.
- Técnicas y Herramientas para la realización de revisiones técnicas del software.
- Testing en ambientes Ágiles.

Unidad 2: Gestión Lean Agile de Productos de Software

Frameworks para escalar Scrum

Scrum limita la cantidad máxima de integrantes del Equipo de Desarrollo. Por eso, surgen frameworks para gestionar productos de mayor envergadura y reducir la complejidad de los ambientes ágiles. Algunos son Nexus, Scale Scrum, LeSS y SAFe.

Framework Nexus

Nexus vincula aproximadamente de tres a nueve equipos Scrum que trabajan sobre un único Product Backlog, con un solo Product Owner, para construir un incremento integrado de producto terminado.

Surge para gestionar las dependencias entre equipos, principalmente las relacionadas con los requerimientos, cuyo alcance e implementación pueden superponerse, y con el conocimiento del dominio, que debe distribuirse para minimizar interrupciones durante el Sprint.

Responsabilidades

- **Nexus Integration Team:** asegura que se produzca al menos un incremento integrado por Sprint y aborda las restricciones que dificultan la integración.
- **Product Owner:** maximiza el valor del producto y gestiona eficazmente el Product Backlog.
- **Scrum Master:** asegura que el framework Nexus se entienda y se aplique según la Guía de Nexus.
- **Miembros del Nexus Integration Team:** ayudan a los equipos a adoptar herramientas y prácticas que mejoren la entrega de incrementos integrados útiles y valiosos.

Eventos de Nexus

Nexus agrega o extiende los eventos de Scrum, con tiempos relacionados con los eventos correspondientes.

- **Nexus Sprint:** los equipos producen un único incremento integrado.
- **Refinamiento entre equipos:** identifica, transparenta y reduce las dependencias; permite prever qué equipo entregará cada elemento. Es continuo.
- **Nexus Sprint Planning:** coordina el trabajo de todos los equipos para un Sprint. Produce los objetivos de Sprint de cada equipo, el objetivo de Nexus, el Nexus Sprint Backlog y los Sprint Backlogs.
- **Nexus Daily Scrum:** inspecciona el progreso hacia el objetivo de Nexus y aborda problemas de integración. Participan representantes de cada equipo.
- **Nexus Sprint Review:** obtiene retroalimentación sobre el incremento integrado terminado y determina adaptaciones futuras. Reemplaza las Sprint Reviews individuales.
- **Nexus Sprint Retrospective:** planifica mejoras en la calidad y eficacia de todo Nexus, inspeccionando equipos, interacciones, procesos, herramientas y la Definición de Terminado.

Artefactos de Nexus

Los artefactos representan trabajo o valor y buscan maximizar la transparencia. Cada uno tiene un compromiso que refuerza el empirismo y el valor de Scrum.

- **Product Backlog:** único listado del trabajo necesario para mejorar el producto. Debe permitir detectar y minimizar dependencias. Su compromiso es el Objetivo del Producto, que describe el estado futuro del producto y constituye un objetivo a largo plazo.
- **Nexus Sprint Backlog:** reúne el Objetivo del Sprint de Nexus y los elementos del Product Backlog de cada equipo. Permite visualizar dependencias y flujo de trabajo, y se actualiza durante el Sprint. Su compromiso es el Objetivo del Sprint de Nexus, un objetivo único que integra el trabajo de los equipos.
- **Integrated Increment:** suma del trabajo integrado completado por Nexus en relación con el Objetivo del Producto. Debe cumplir la Definición de Terminado, que establece la calidad requerida. Solo está terminado cuando está integrado, es valioso y utilizable. Todos los equipos deben respetarla; pueden ser más estrictos, pero nunca menos rigurosos.

1. Filosofía Lean

La filosofía Lean nace en Japón en los años 40 con Toyota y busca maximizar el valor generado al cliente con el mínimo consumo de recursos. Todo esfuerzo que no cree valor se considera desperdicio, por lo que se busca reducir los esfuerzos en crear artefactos que no agreguen valor al producto.

Lean conduce a una visión integrada de la cultura y la estrategia para atender al cliente final con alta calidad, bajo costo y tiempo de entrega, produciendo exactamente lo que el cliente quiere, cuando lo quiere y dónde lo quiere, a un costo mínimo y precio justo. El cliente determina si el servicio o producto tiene valor.

Para esto utiliza el método **Just in Time**, que permite entregar el producto justo a tiempo, poco antes de que lo usen y en la cantidad justa y necesaria.

2. Los 7 principios Lean

Los principios Lean se adaptaron al desarrollo de software a partir de los principios originales de Lean, debido a que el software es un producto intangible.

- **Eliminar desperdicios:** eliminar todo paso o parte del proceso que no contribuya a generar valor. Se busca evitar funcionalidades en exceso, retrabajo y artefactos obsoletos.
- **Amplificar el aprendizaje:** fomentar la transparencia y transformar el conocimiento implícito en explícito, fortaleciendo al equipo mediante el intercambio de experiencias y conocimientos.
- **Embeber la integridad conceptual:** la calidad del producto no se negocia y la simplicidad es un factor clave. El producto debe satisfacer las expectativas y necesidades del cliente.
- **Diferir compromisos hasta el último momento responsable:** postergar las decisiones hasta tener la mayor información necesaria, pero antes de que sea demasiado tarde.
- **Dar poder al equipo:** otorgar libertad de acción y poder de decisión a un equipo multifuncional.
- **Ver el todo:** comprender el producto y el valor agregado, alineando los objetivos particulares de las partes con los objetivos globales.
- **Entregar lo antes posible:** entregar software funcionando lo más rápido posible y que sea útil para el cliente, antes de que cambien sus necesidades.

3. Los 7 desperdicios de Lean en software

Los desperdicios son actividades que consumen recursos sin generar valor para el producto.

1. **Características extras:** desarrollar funcionalidades de más o especificar anticipadamente requerimientos que pueden quedar obsoletos, incompletos o erróneos.
2. **Trabajo a medias:** realizar trabajo sin terminar, que debe retomarse posteriormente.
3. **Proceso extra:** ejecutar pasos que no aportan valor al producto.
4. **Búsqueda de información:** la falta de información suficiente o accesible dificulta la trazabilidad, la identificación del impacto de los cambios y la resolución de errores.
5. **Defectos:** errores que pasan de una etapa a otra y generan retrabajo. El aseguramiento de calidad permite evitarlos.
6. **Esperas:** tiempo que una persona debe esperar para realizar su trabajo porque depende de otra persona o de una aprobación.
7. **Cambios de tareas (Multitasking):** cambiar constantemente de tarea genera pérdidas de tiempo. Kanban limita el WIP para reducir el multitasking.

4. Relación entre Lean y Agile

Lean es previo al Manifiesto Ágil. Algunos principios de Agile heredan fundamentos de Lean. Ambos enfoques están orientados al cliente y buscan proveer el máximo valor posible. También promueven equipos motivados, flexibilidad para adaptarse a los cambios, productos de calidad y mejora continua.

5. Kanban y su relación con Lean

Según el texto, Kanban se relaciona con Lean porque busca reducir desperdicios y mejorar el flujo de trabajo. Para reducir los cambios de tareas y el multitasking, utiliza la limitación del WIP (Work in Progress), es decir, del trabajo en progreso.

En Kanban, limitar el WIP permite reducir el multitasking. Esto se relaciona con el método Just in Time (JIT).

Métricas ágiles y en otros enfoques

¿Qué es una métrica?

Una **métrica** es un número que representa determinados atributos de un proceso, producto o proyecto y permite una medición objetiva de la realidad. La métrica no es la escala, sino la cantidad de cierta escala.

Toda métrica tiene un costo asociado, por lo que debe realizarse un análisis de costo-beneficio. Las métricas no deben utilizarse para evaluar o castigar a las personas.

¿Para qué medir?

Para controlar y supervisar proyectos, estimar y predecir, evaluar costos y mejorar procesos, proyectos y productos.

Dominios de métricas

- **Proceso:** mejora los procesos de software a largo plazo. Son públicas, generales y responsabilidad del Ingeniero de Procesos.
- **Proyecto:** mide costos, esfuerzo, estimaciones y tiempo. Es responsabilidad del Líder de Proyecto y permite mejorar la planificación, reducir riesgos y controlar la calidad.
- **Producto:** mide lo construido, como el código, las pruebas y la calidad. Es responsabilidad del equipo de desarrollo y Testing.

Enfoque tradicional

Se basa en proyectos definidos y utiliza métricas de proceso, proyecto y producto. Las métricas básicas son tamaño del producto, esfuerzo, calendario y defectos, relacionadas con la triple restricción: esfuerzo, alcance y tiempo.

Ambientes ágiles

La mejor métrica de progreso es el software funcionando. Se mide lo necesario para aportar valor al cliente, y las métricas de un equipo no son extrapolables a otros equipos o proyectos.

- **Velocidad:** puntos de historia completados y aceptados por el Product Owner durante un Sprint. Permite analizar la estabilidad del equipo.
- **Capacidad:** compromiso de trabajo del equipo para un Sprint, estimado al inicio en horas ideales o Story Points. Sirve para planificar.
- **Running Tested Features (RTF):** cantidad de funcionalidades testeadas que están funcionando. No considera su complejidad.

Las métricas de defectos son necesarias en todos los ambientes.

Lean (Kanban)

Se enfoca en el proceso y el flujo de trabajo continuo, midiendo el tiempo que tardan las funcionalidades.

- **Lead Time o Elapsed Time:** desde que el cliente solicita algo hasta que se entrega.
- **Cycle Time:** desde que comienza el trabajo hasta que se entrega, excluyendo la espera en el Backlog. Cycle Time = Lead Time − tiempo en el Backlog.
- **Touch Time:** tiempo de trabajo efectivo, sin contar las esperas.
- **Eficiencia del ciclo de proceso:** relación entre el tiempo de trabajo efectivo y el tiempo total de entrega. Fórmula: Touch Time / Lead Time. Cuanto más cercano a 1, más eficiente es el proceso.

Resumen para memorizar

- **Tradicional:** esfuerzo, tiempo, costos y riesgos.
- **Ágil:** velocidad, capacidad y Running Tested Features.
- **Lean:** Lead Time, Cycle Time, Touch Time y eficiencia del proceso.

Unidad 3: Gestión de software como producto

Gestión de configuración en ambientes ágiles

En metodologías ágiles, la gestión de configuración permite el seguimiento y la coordinación del desarrollo, en lugar de controlar a los desarrolladores. Busca responder a los cambios y mantener la integridad y calidad del producto. Es responsabilidad de todo el equipo y sus actividades se integran con las tareas del Sprint. Las auditorías no son propias de los ambientes ágiles, cuyos procesos tradicionales se flexibilizan.

Prácticas continuas

- **Continuous Integration (CI):** integrar el código a un repositorio compartido varias veces al día. Cada integración se verifica con pruebas automatizadas para detectar errores tempranamente. Busca que el código compile y sea de calidad.
- **Continuous Delivery (CD):** construir el software para que pueda liberarse en producción en cualquier momento. Automatiza las pruebas de aceptación, pero la liberación depende de la decisión del Product Owner. Requiere integración continua y un software siempre entregable.
- **Continuous Deployment:** desplegar el producto en producción sin intervención humana, mediante pipelines con pasos ordenados.

Estrategias de despliegue

- **Canary Deployment:** lanzar una actualización primero a una pequeña parte de los usuarios y, una vez aceptada, extenderla al resto. Limita el impacto de las fallas y permite observar el uso real.
- **Blue-Green Deployment:** transferir gradualmente el tráfico de una versión anterior (azul) a una nueva (verde). La versión anterior queda disponible para una posible reversión.

Diferencia clave: CI integra y prueba el código; CD mantiene el software listo para liberar, con decisión humana; Deployment despliega automáticamente en producción.

Unidad 4: Aseguramiento de Calidad de Proceso y de Producto

Conceptos generales sobre calidad

La **calidad del software** es el cumplimiento de los requerimientos funcionales y de performance, de los estándares de desarrollo y de las características implícitas esperadas. Es un concepto subjetivo, relacionado con las necesidades y expectativas de cada persona.

La **gestión de calidad** consiste en establecer estándares a nivel organizacional, verificar los procesos y elaborar un plan de calidad para cada proyecto. El **aseguramiento de calidad** define procesos y estándares para obtener productos de alta calidad. Su objetivo es prevenir errores, a diferencia del

testing, que permite detectarlos posteriormente. El equipo de QA debe ser independiente del equipo de desarrollo para mantener una perspectiva objetiva.

La calidad del software debe satisfacer las expectativas del cliente, del usuario, de la gerencia, del equipo de desarrollo y mantenimiento y de otros interesados. Para lograrla, debe estar incorporada desde los requerimientos y ser responsabilidad de todos los involucrados. Se necesita capacitación, liderazgo, métricas y planificación. Además, la prevención es menos costosa que la corrección, ya que esta última implica retrabajo.

Trabajar con calidad permite competir en el mercado, retener clientes, incrementar beneficios y reducir costos y retrabajo. Puede analizarse desde **cinco visiones**: usuario, según sus expectativas; producto, según sus requerimientos; proceso, según el valor que aporta; valor, según la relación costo-beneficio; y trascendental, mediante objetivos difíciles de alcanzar que impulsan la mejora continua.

Importancia de trabajar para y con Calidad. Ventajas y Desventajas

Trabajar para y con calidad es fundamental en la producción de software porque constituye un aspecto competitivo que permite sobrevivir en el mercado internacional. Las empresas certifican estándares de calidad de proceso, no de producto, para poder competir. Además, la calidad retiene a los clientes e incrementa los beneficios, ya que implica cumplir con sus expectativas. También permite alcanzar un equilibrio entre costo y efectividad, reduciendo los costos y el retrabajo, porque se asume que las fallas serán menores.

Por otro lado, trabajar con baja calidad aumenta el costo del retrabajo, debido a las correcciones de errores y a los reiterados intercambios entre las actividades de testing y desarrollo. El testing también es una actividad costosa y no puede cubrir el 100 % de las pruebas por cuestiones de tiempo y costos. Por eso, es necesario buscar una cobertura óptima teniendo en cuenta el costo-beneficio.

En conclusión, la prevención es mejor que la corrección, porque prevenir es menos costoso que corregir, debido al alto costo asociado al retrabajo.

Actividades relacionadas con el Aseguramiento de la Calidad del Software

Las actividades de aseguramiento de la calidad del software buscan alcanzar los niveles requeridos de calidad mediante estándares y procesos apropiados, promoviendo una cultura en la que la calidad sea responsabilidad de todo el equipo.

La **administración de calidad** comprende tres actividades: aseguramiento de calidad, que define estándares, procesos y procedimientos; planificación de calidad, que determina las actividades, los estándares aplicables y los atributos de calidad del producto; y control de calidad, que ejecuta lo planificado y verifica el cumplimiento de los estándares.

El **control de calidad** se realiza mediante revisiones de calidad, que permiten encontrar problemas potenciales, y evaluaciones automáticas y mediciones. También se evalúan la planificación del proyecto, los requerimientos, el diseño, la programación, la integración, las pruebas y el control del proyecto.

Principales Modelos de Calidad existentes (CMMI – SPICE – ISO)

CMMI

El **CMMI (Capability Maturity Model Integration)** es un modelo de calidad para la mejora y evaluación de los procesos de desarrollo, mantenimiento y operación de sistemas de software. Es un modelo de referencia que establece objetivos y buenas prácticas, pero permite que la organización defina cómo implementarlos.

Tiene dos representaciones: **por etapas**, que evalúa la madurez de la organización como un todo mediante cinco niveles (1 al 5), y **continua**, que evalúa la capacidad de cada área de proceso mediante seis niveles (0 al 5).

Los niveles de madurez por etapas son: **Inicial**, con procesos impredecibles; **Administrado**, con procesos planificados y controlados; **Definido**, con procesos documentados y estandarizados; **Administrado cuantitativamente**, con control mediante métricas; y **Optimizado**, orientado a la mejora continua.

Las áreas de proceso son conjuntos de prácticas relacionadas que, implementadas en conjunto, permiten alcanzar objetivos y lograr mejoras significativas. La acreditación se obtiene mediante la evaluación del proceso y de su aplicación en los proyectos.

SPICE

El modelo **SPICE (Software Process Improvement Capability Evaluation)** es un modelo creado para la mejora de procesos de software, basado en la norma ISO 15504. Permite evaluar los procesos de desarrollo de software mediante seis niveles de capacidad.

Es un modelo dual, teórico, compuesto por dos partes: el modelo de calidad y los modelos de mejora de procesos SPICE e IDEAL, que se utilizan para crear proyectos de mejora en una organización. El resultado es un proceso definido que luego se utiliza para desarrollar proyectos.

ISO

Los modelos y estándares **ISO** sirven como referencia para definir procesos de trabajo y evaluar su cumplimiento. La organización puede utilizar modelos de mejora continua para mejorar sus procesos y modelos de evaluación para determinar el grado de adherencia del proceso al modelo de referencia.

La **ISO 9001** establece requisitos para el sistema de gestión de calidad. Una auditoría permite identificar las diferencias entre el proceso definido por la organización y lo que establece la norma.

En cuanto a la calidad del producto, el resumen menciona el **Modelo de McCall** y la **ISO 25010**, que permiten evaluar características de calidad del software, especialmente los requisitos no funcionales. Como los requerimientos varían según cada producto, no existe un modelo que certifique por sí solo el nivel de calidad de un software determinado.

Auditorías al Software: Auditorías de Proyecto y Auditorías al Grupo de Calidad

Las **auditorías de software** permiten evaluar objetivamente el producto y verificar su consistencia durante el proceso de desarrollo. Se utilizan para comprobar que el software cumpla con los requerimientos y que el código y la documentación sean consistentes.

La auditoría de configuración funcional verifica que el producto cumpla con los requerimientos especificados en la ERS. La auditoría de configuración física comprueba que el código coincida con la documentación técnica que lo describe. También existen auditorías externas de aseguramiento de calidad, realizadas por personal externo a la organización.

Sus beneficios son detectar desviaciones, verificar el cumplimiento de los requerimientos y estándares, asegurar la consistencia entre el código y la documentación, y permitir que los problemas encontrados sean corregidos antes de entregar el software.

La **auditoría de proyecto** verifica que el proyecto se haya ejecutado siguiendo el proceso definido, adaptado al contexto particular. Para ello, se recopilan evidencias y se contrastan con la documentación, como la ERS, la arquitectura y el diseño. Su objetivo es verificar objetivamente la consistencia del producto durante el desarrollo, según lo establecido en el PACS.

La **auditoría de configuración funcional** valida que el producto cumpla con los requerimientos especificados en la ERS. Compara el software construido y su documentación con esos requerimientos, para asegurar que el código implemente correcta y completamente las capacidades funcionales descriptas. QA también debe verificar que la matriz de trazabilidad esté actualizada.

La **auditoría de configuración física** verifica que el ítem de configuración cumpla con la documentación técnica que lo describe. Compara el código con la documentación de soporte para asegurar que la documentación que se entregará sea consistente y describa correctamente el código desarrollado. El software solo podrá entregarse cuando se hayan corregido las desviaciones encontradas.

Proceso de Auditorías

El proceso de auditoría comprende cuatro etapas:

1. **Preparación y planificación:** el auditado solicita la auditoría y, junto con el auditor, define la fecha, el alcance, los objetivos y los participantes. También se establecen métricas, como el esfuerzo, la duración y la cantidad de desviaciones.
2. **Ejecución:** se realiza una reunión de apertura para informar cómo será el proceso. Luego, el auditor formula preguntas, solicita evidencias, revisa la documentación y completa una lista de verificación (checklist).
3. **Análisis y reporte de resultados:** se evalúan los resultados, se realiza una reunión de cierre y se entrega el reporte final. El auditado analiza las conclusiones y puede manifestar su desacuerdo con algunas prácticas.
4. **Seguimiento:** se analizan las desviaciones y se prepara un plan de acción que se envía al auditor para su revisión y aprobación. El auditor puede realizar un seguimiento hasta considerar que las desviaciones fueron resueltas.

Las herramientas y técnicas de auditoría son:

- **Checklists:** contienen preguntas tipo para garantizar que, independientemente de quién realice la auditoría, se controle lo mismo como mínimo. El auditor puede agregar más preguntas.
- **Muestreo:** consiste en seleccionar una muestra representativa de los productos o procesos que se van a auditar.
- **Revisión de registros:** permite revisar los registros relacionados con el proceso auditado.
- **Herramientas automatizadas:** se utilizan para apoyar las tareas de auditoría.

Los resultados de una auditoría pueden ser **buenas prácticas**, cuando se supera lo definido; **desviaciones**, cuando algo no se hizo o no se hizo según el proceso definido, clasificadas en No Adecuado o Necesita Mejora; y **observaciones**, cuando se detectan prácticas que podrían generar riesgos, pero que no constituyen desviaciones y no requieren un plan de acción.

El reporte de auditoría debe informar las desviaciones que generen disconformidad del producto respecto de sus requerimientos y las desviaciones respecto del proceso definido o de los requerimientos documentados.

Técnicas y Herramientas para la realización de revisiones técnicas del software

La **verificación y validación** son procesos que abarcan todo el ciclo de vida del software. Comienzan con las revisiones de los requerimientos y continúan con las revisiones del diseño, las inspecciones del código y las pruebas.

La **validación** responde a la pregunta: ¿Estamos construyendo el producto correcto? La **verificación** responde a: ¿Estamos construyendo el producto correctamente?

Una **falla** es un error en un producto de trabajo, entendido como la salida de cualquier actividad correspondiente al ciclo de vida de desarrollo.

Las **revisiones técnicas del software (Peer Review)** son actividades realizadas por un colega para mejorar la calidad del software mediante la detección temprana de errores en cualquier producto de trabajo, como el código, los requerimientos, el diseño, la arquitectura, los riesgos, las estimaciones y los planes.

Es un proceso estático de verificación y validación que no corrige errores, sino que busca evitar el retrabajo y motivar a realizar un mejor trabajo. Nunca se pone en juicio al autor del artefacto, sino al artefacto en sí, promoviendo una cultura de trabajo que brinde apoyo y no busque culpables.

Sus **ventajas** son que pueden descubrirse muchos errores, inspeccionarse versiones incompletas y considerarse otros atributos de calidad. Sus **desventajas** son que es difícil introducir las inspecciones formales, que inicialmente aumentan los costos y que requieren tiempo para organizarse.

Existen dos tipos de revisiones:

- **Formales:** tienen un proceso definido con roles, como las inspecciones de Fagan y de Gilb.
- **Informales:** no tienen un proceso definido, como el Walkthrough o recorrido.

El **Walkthrough** es una técnica de análisis estático en la que un diseñador o programador guía al equipo a través de un producto de software. Los participantes formulan preguntas y comentan posibles errores, violaciones de estándares y otros problemas. Sus objetivos son la mínima sobrecarga, la capacitación de desarrolladores y el rápido retorno.

Las **inspecciones formales** cuentan con roles definidos, utilizan un checklist, toman métricas y elaboran un reporte para analizar los defectos encontrados. Buscan descubrir errores, verificar que el software alcance sus requerimientos y garantizar que haya sido construido de acuerdo con ciertos estándares.

Calidad de Producto: Planificación de pruebas para el software

La **planificación de pruebas** determina cómo se incluirá el Testing en el plan del proyecto y cómo será el Plan de Pruebas. Este debe establecer los riesgos y objetivos del testing, la estrategia, los recursos, los criterios de aceptación, los entornos que se utilizarán, quién realizará cada prueba y cuándo.

El proceso de pruebas se lleva a cabo durante todo el ciclo de vida del producto y comprende distintas etapas. El **Testing Ad Hoc** se realiza sin un proceso definido ni registros del paso a paso de lo que se probó.

Niveles de pruebas:

- **Pruebas unitarias:** prueban componentes individuales.

- **Pruebas de integración:** verifican el funcionamiento de dos o más componentes que interactúan entre sí mediante sus interfaces.
- **Pruebas de sistema o de versión:** prueban una versión del producto de manera sistemática, siguiendo casos de prueba, para encontrar defectos reproducibles. Abarcan requerimientos funcionales y no funcionales.
- **Pruebas de aceptación de usuario:** verifican que el producto cumpla con lo que solicitó el usuario. Buscan que el cliente se familiarice con el producto y genere comodidad y confianza.

Las pruebas de aceptación incluyen las **pruebas alfa**, realizadas por el usuario en un ambiente de laboratorio, y las **pruebas beta**, realizadas en ambientes de trabajo reales.

Las técnicas de testing permiten diseñar casos de prueba para detectar defectos en el software. Según el resumen, se distinguen dos enfoques principales:

- **Técnicas basadas en especificaciones:** incluyen la partición de equivalencias, que divide las condiciones externas de entrada y salida en subconjuntos de valores posibles que producen resultados equivalentes, y el análisis de valores límite, que utiliza los límites de las clases de equivalencia para definir los casos de prueba.
- **Métodos basados en la experiencia:** utilizan los conocimientos del tester para determinar las entradas y analizar los resultados. Incluyen la adivinanza de defectos, basada en la intuición y experiencia para identificar situaciones propensas a errores, y el testing exploratorio, en el que el tester aprende a manejar el sistema y genera nuevas pruebas a partir de su experiencia y creatividad.

Para elaborar los casos de prueba se deben considerar entradas válidas y no válidas, establecer prioridades y definir precondiciones concretas. Los casos deben ser reproducibles y permitir detectar y reportar defectos.

Testing en ambientes ágiles

El testing de software es una de las tareas del aseguramiento de calidad, junto con el control de calidad del proceso y del producto. La calidad del software se logra a lo largo de todo el desarrollo, porque detectar errores en etapas tempranas es menos costoso que detectarlos después.

La **gestión de configuración (SCM)** es el punto de partida para realizar tareas de aseguramiento de calidad, ya que permite determinar la configuración de los ítems, mantener la trazabilidad, controlar cambios, realizar auditorías del producto y generar reportes de estado. QA agrega el control de calidad del proceso. Existen estándares, como ISO y CMMI, que permiten acreditar la calidad del proceso; la calidad del producto no es estandarizable de la misma manera porque los requerimientos varían según cada caso.

En las pruebas de aceptación, el objetivo es verificar que el producto cumpla con lo que solicitó el usuario y que este se familiarice con el producto, generando comodidad y confianza. Se distinguen las pruebas alfa, realizadas por el usuario en un ambiente de laboratorio, y las pruebas beta, realizadas en ambientes de trabajo reales.

En Agile, además del usuario, deben estar presentes todos los miembros del equipo durante la Sprint Review.